PROJET AWD · VALIDATION INTERMÉDIAIRE · SÉANCE 4

AgriMeteo

Plateforme intelligente de météo et d'irrigation agricole

Rayen · Mohamed · Ilef · Ala — [classe-groupe]

Idée générale et problématique

Problématique

Irrigation décidée sans données météo fiables : gaspillage d'eau, stress hydrique et pertes de rendement.

Solution

AgriMeteo croise la météo locale et les parcelles pour recommander l'irrigation et alerter en temps utile.

Agriculteur

Gère ses parcelles, consulte la météo, reçoit alertes et recommandations.

Administrateur

Gère les utilisateurs et supervise la plateforme.

API météo externe

Source des prévisions et des données historiques.

Principales fonctionnalités

Comptes

Inscription, connexion et authentification sécurisée.

Parcelles

Création et suivi : superficie, type de culture.

Météo

Prévisions et historique par localisation.

Irrigation

Besoin en eau selon météo, culture et superficie.

Alertes

Gel, canicule, forte pluie, vent.

Notifications

Envoi des alertes et recommandations d'irrigation.

Architecture globale



Communication synchrone : REST via Gateway · Asynchrone : événements · Base par service

Microservices et bases de données

Service	Responsabilité	Base
User (commun)	Utilisateurs, inscription, authentification	PostgreSQL
Weather	Récupération et stockage des données météo	MongoDB
Field Management	Parcelles, superficies, types de cultures	PostgreSQL
Irrigation	Analyse météo et besoins en irrigation	PostgreSQL
Notification	Alertes météo et recommandations	MongoDB

Technologies et sécurité

Backend

Spring Boot, Spring Cloud Gateway, Eureka

Frontend

Angular

Données

PostgreSQL, MongoDB

Communication

REST, OpenFeign, RabbitMQ

Sécurité

JWT, rôles, Gateway comme point d’entrée

DevOps

Docker, Docker Compose, GitHub

Interactions entre services

Synchrone (REST)

Irrigation → Field : GET /fields/{id}
Irrigation → Weather : GET /weather/forecast
Gateway → User : validation du JWT

Asynchrone (événements)

Weather publie WeatherAlertDetected
Irrigation publie IrrigationRecommended
User publie UserRegistered
Notification consomme les trois

Contrats d’API (principaux)

Service	Endpoints	Codes
User	POST /auth/register · POST /auth/login	201, 200, 400, 401, 409
Weather	GET /weather/current · /forecast · /history	200, 404, 503
Field	GET/POST /fields · PUT/DELETE /fields/{id}	200, 201, 204, 404
Irrigation	GET /irrigation/recommendations?fieldId=	200, 404
Notification	GET /notifications · PATCH /{id}/read	200, 404

Équipe, répartition et dépôt Git

Étudiant	Microservice
Rayen	Weather Service
Mohamed	Field Management Service
Ilef	Irrigation Service
Ala	Notification Service
Tous	User Service (commun)

Dépôt : **agrimeteo-[classe]-[groupe]** → Backend/ · Frontend/ · Documentation/ (Presentation, Diagrams) · README.md
Gestion de projet : [GitHub Projects / Trello]

Planning prévisionnel et risques

Planning

S4 : conception et dépôt
Puis : User + Gateway, services métier, événements, frontend, tests et déploiement

Risques

Quota ou panne de l'API météo
Cohérence entre services
Intégration tardive
Retards sur le service commun